Árboles de Modelos

Modelos Avanzados de Aprendizaje Automático I

En el ejercicio anterior se desarrollaron funciones que permiten el entrenamiento de Árboles de Regresión. Estos constituyen una de las herramientas más utilizadas para problemas de regresión. Sin embargo, presentan una limitación importante: en cada hoja del árbol se asume una predicción constante, lo cual puede resultar demasiado restrictivo cuando la relación entre las variables de entrada y la salida es más compleja.

Los Árboles de Modelos (*Model Trees*) surgen como una extensión natural de los Árboles de Regresión para solventar esta limitación. La idea fundamental consiste en sustituir las predicciones constantes en las hojas por modelos más expresivos. En su formulación original (algoritmos M5 y M5') estos modelos son modelos lineales; sin embargo, en este ejercicio se posibilitará el uso de cualquier tipo de modelo, incluyendo modelos lineales, máquinas de vectores de soporte o modelos simbólicos. De esta forma, el árbol no solo segmenta el espacio de entrada, sino que también ajusta un modelo específico en cada región.

Este enfoque, por lo tanto, combina dos ideas clave:

- Particionado del espacio (propio de los árboles de decisión).
- Modelización local (propia de modelos de regresión más sofisticados).

Como resultado, los árboles de modelos son capaces de capturar relaciones no lineales complejas manteniendo una estructura interpretable.

Para implementar esto, se utilizará la misma estructura del ejercicio anterior. Como se indicó entonces, los modelos internos, tanto en hojas como en nodos, son de tipo *Any*, posibilitando que se pueda tener cualquier tipo de modelo. Para ello, se creaba una interfaz común a todos ellos, utilizando la función *predict*, o desarrollándola si esta no existía para un modelo concreto. De hecho, en el ejercicio anterior se desarrolló la función *predict* para poder usarla con dos tipos de modelos:

- Constantes.
- Árboles de Regresión, con una implementación que sigue siendo funcional en este ejercicio como Árboles de Modelos.

Según la implementación seguida en el ejercicio anterior de la función *predict* para un Árbol de Regresión, al evaluar una hoja, esta devolvía el resultado de llamar a *predict* del modelo que contenía. Por lo tanto, es necesario definir la función *predict* en los modelos que no la tengan

definida. En este ejercicio, se utilizarán también los siguientes modelos de Regresión para constituir los modelos internos del árbol:

- Modelos lineales. Utilizados en los algoritmos originales M5 y M5' de los Árboles de Modelos. En este ejercicio, un modelo lineal se define como una tupla de dos valores, donde el primero es el término libre, y el segundo es un vector con los coeficientes de regresión, de la siguiente manera:

```
LinearModel = Tuple{Real, AbstractArray{<:Real,1}}
```

  Al estar definido en este ejercicio, será necesario definir la función *predict*.
- SVR o Máquinas de Vectores de Soporte para Regresión. Suponen una extensión de las Máquinas de Vectores de Soporte, y, en este ejercicio, se utilizará la variante *EpsilonSVR*, por ser la más similar al modelo original. Se provee de una implementación de la función *predict* en su implementación EpsilonSVR, que se puede utilizar a través de la librería MLJ mediante:

```
EpsilonSVR = MLJ.@load EpsilonSVR pkg=LIBSVM verbosity=0
```

  Una vez cargado, la función *predict* es la siguiente:

```
predict(model::Machine{MLJLIBSVMInterface.EpsilonSVR,     MLJLIBSVMInterface.EpsilonSVR,
true}, inputs::AbstractArray) = MLJ.predict(model, inputs)
```

- DoME, que permite obtener modelos simbólicos. Será necesario definir la función *predict*, puesto que la librería utiliza una función de nombre distinto para realizar predicciones.

En este ejercicio, se completarán las funciones del ejercicio anterior para desarrollar modelos más complejos basados en árboles para resolver problema de regresión. Además de la función *predict*, se desarrollarán nuevas versiones de las funciones definidas en el ejercicio anterior, así como funciones relativas a entrenar modelos de Regresión. Concretamente, se pide realizar las siguientes funciones:

- *predict*. Función análoga a la desarrollada en el ejercicio anterior, pero, en este caso, para dos tipos nuevos de modelos: modelos lineales y modelos simbólicos desarrollados con el algoritmo DoME (librería SymDoME). Por lo tanto, será necesario desarrollar dos funciones, que reciben los mismos dos argumentos (*model* y *inputs*), cuyo tipo de salida deberá ser el mismo (vector de valores reales), y cuya firma sólo cambiará en el tipo del primer argumento *model*:
    - Si es de tipo *LinearModel*, el modelo será un modelo lineal. Para aplicarlo a la matriz *inputs*, esto se realizará de una forma muy sencilla mediante un producto

matricial de la matriz de entradas *inputs* y el vector de coeficientes del modelo lineal, tras lo cual se suma al resultado (vector) el término libre del modelo lineal.

- Si es de tipo *SymDoME.Tree*, el modelo será una expresión simbólica desarrollada con el algoritmo DoME. En este caso, se puede llamar a la función *evaluateTree* pasando como primer hijo el modelo y como segundo hijo la matriz de entradas *inputs* para evaluar el modelo. Hay que tener en cuenta que, cuando el modelo devuelve el mismo valor numérico para todas las instancias, esta función lo devuelve como un valor escalar en lugar de un vector con un elemento por instancia. Este caso tendrá que ser contemplado, y este valor numérico deberá ser convertido a un vector de valores reales, lo cual se puede hacer fácilmente con la función *fill*.

Para el desarrollo de estas funciones sobrecargadas no se permite el uso de bucles.

- *trainRegressionModel*. Esta función permitirá, de forma genérica, entrenar un modelo de Regresión. Esta es una función sobrecargada, es decir, en realidad son varias funciones o métodos. En este ejercicio, se han reunido en dos grupos distintos que se distinguen, como siempre, en los parámetros que reciben. El primer grupo comprende la función que recibe los siguientes parámetros:
  - *configuration*, de tipo *Dict*, con la configuración del modelo a entrenar. La configuración contiene distintas claves con los hiperparámetros que tiene el modelo a entrenar. Todas las claves deberán ser de tipo *String*. Una de estas claves será "modelType", que describe de qué tipo será el modelo a entrenar, y podrá tener uno de los siguientes valores, todos de tipo *Symbol*:, para denotar el modelo a entrenar:
    - :constant
    - :linear
    - :DoME
    - :SVR
    - :ModelTree
    - :KNN. Este modelo se entrenará en el siguiente ejercicio.

    Según el tipo de modelo, este argumento de configuración tendrá más o menos claves que almacenen los hiperparámetros necesarios de cada modelo. Por ejemplo, para los casos de modelos :constant y :linear no se necesitan más hiperparámetros, para el algoritmo :DoME se necesitará el campo

"maximumNodes" con el número máximo de nodos, etc.

- *dataset*, de tipo *Batch*, con el conjunto de entrenamiento.

Una opción posible para implementar esto es que, en el interior de esta función, haya una serie de comprobaciones según el campo "modelType" y se entrene el modelo correspondiente. Esto obligaría a modificar esta función en caso de querer añadir nuevos modelos a entrenar.

De forma alternativa, en este ejercicio este proceso se realiza mediante una serie de funciones con el mismo nombre, una para cada método a entrenar. Este es el segundo grupo de funciones que se comentaba más arriba. Para llamarlas, estas recibirán como primer argumento el tipo del modelo (del campo "modelType"), y se ejecutará solamente aquella definida para ese tipo.

Por lo tanto, esta función en realidad no realiza el entrenamiento de ningún modelo, sino que delega esta acción en la función correspondiente. Lo único que hará será llamar a la función *trainRegressionModel* descrita a continuación, pasándole 3 argumentos. El primero será el tipo del modelo, sacado de la clave "modelType" de la configuración. Para que esto se pueda usar para que se llame a una función distinta según el valor, es necesario convertir este valor en un tipo, lo cual se puede hacer con la función *Val*. El segundo y el tercer argumento serán, respectivamente, la configuración y el *dataset*.

Para el desarrollo de esta función no se permite el uso de bucles.

- *trainRegressionModel*. Segundo grupo de funciones, estas tienen el código para entrenar cada modelo, uno por función. Como en toda función sobrecargada, será llamada la función según el número y tipo que tienen sus parámetros. En todos los casos el segundo y el tercer parámetro serán respectivamente, *configuration*, de tipo *Dict*, y *dataset*, de tipo *Batch*. El tipo del primer parámetro, que se corresponderá con cada una de las funciones, podrá ser uno de los siguientes:
  - *Val{:constant}*. En este caso, esta función se usará para entrenar un modelo constante, que será igual al valor promedio de las salidas deseadas. Por lo tanto, esta función deberá devolver un valor escalar del mismo tipo que los elementos del vector de salidas deseadas.
  - *Val{:linear}*. En este caso, esta función se usará para entrenar un modelo lineal. A pesar de que existen formas más robustas de entrenarlo, para simplificar el código esto se hará mediante el operador \, de la siguiente manera:
    - Se crea una matriz uniendo una primera columna con unos a la matriz con las entradas del *dataset*. Esta nueva matriz deberá mantener el mismo

tipo de elemento que la matriz de entradas del *dataset*. Para hacer esto, puedes ser útiles las funciones *hcat* y *ones*.

- Se realizará la operación \ entre esta matriz y el vector de salidas deseadas, teniendo como resultado un vector de coeficientes, donde el primero será el término libre.
- Se devuelve un objeto de tipo *LinearModel*, tomando la información del vector anterior. Los elementos de este objeto (el escalar con el término libre y los coeficientes) deberán ser del mismo tipo que las entradas y salidas deseadas del *dataset*.

- *Val{:DoME}*. En este caso, esta función entrenará el método de Regresión Simbólica DoME. Esto se hace mediante una llamada a la función dome indicando, como primer argumento, la matriz de entradas (se puede extraer mediante una llamada a la función *batchInputs*), como segundo argumento el vector de salidas deseadas (función *batchTargets*), y como argumento opcional con el *keyword maximumNodes* el valor de la clave "maximumNodes" de la configuración, con el número máximo de nodos. El resultado de esta llamada devuelve una tupla con varios valores, siendo el primero el modelo buscado, y lo que debe ser devuelto, de tipo *SymDoME.Tree*.
- *Val{:SVR}*. Esta función entrenará un SVR, la variable de configuración contendrá los valores de los hiperparámetros. Estos son los mismos que para un SVM, y uno adicional, "epsilon", con el ancho del margen, que contiene el error permitido para cada instancia en la regresión. El proceso para entrenar un SVR es similar al de entrenar un SVM. En primer lugar, es necesario llamar a la función *EpsilonSVR* indicando cada uno de los hiperparámetros, forzando el tipo de cada hiperparámetro al tipo concreto que debe recibir, en caso contrario podría dar un error. La siguiente tabla muestra, en la primera columna, el nombre que recibe cada hiperparámetro en la llamada a la función *EpsilonSVR*. La segunda columna contiene el nombre de la clave en la configuración (de tipo *Dict*). La tercera tiene el tipo con el que es necesario pasar el hiperparámetro, y la cuarta columna el valor por defecto que debe tomar, en caso de que la configuración (de tipo *Dict*) no contenga esa clave.

Nombre del hiperparámetro	Clave	Tipo	Valor por defecto
kernel	“kernel”	String	“rbf”
epsilon	“epsilon”	Float64	0.1
cost	“C”	Float64	1
gamma	“gamma”	Float64	1 / <num. atributos>
degree	“degree”	Int32	3
coef0	“coef0”	Float64	0

Los posibles valores para la clave “kernel” a utilizar son los siguientes, junto con los *kernels* asociados:

- “linear”: LIBSVM.Kernel.Linear
- “rbf”: LIBSVM.Kernel.RadialBasis
- “poly”: LIBSVM.Kernel.Polynomial
- “sigmoid”: LIBSVM.Kernel.Sigmoid

Por ejemplo, una llamada a la función *EpsilonSVR* podría ser la siguiente:

```
model = EpsilonSVR(
        kernel = “poly”
        epsilon = Float64(0.1),
        cost    = Float64(1),
        gamma   = Float64(0.1),
        degree  = Int32(3),
        coef0   = Float64(0)
)
```

Al igual que para entrenar un SVM, el resultado de esta llamada se usará como primer argumento de una llamada a la función *machine*. La diferencia está en que, en un problema de Regresión, no será necesario convertir el tercer argumento de forma categórica. Posteriormente se llamará a la función *fit!* para entrenar el modelo, y se devolverá el modelo entrenado, de la siguiente manera:

```
mach = machine(model, MLJ.table(batchInputs(dataset)), batchTargets(dataset));
MLJ.fit!(mach, verbosity=0)
return mach
```

De esta forma, se permite entrenar nuevos modelos simplemente definiendo las funciones correspondientes, como se hará más adelante.

Para el desarrollo de estas funciones sobrecargadas no se permite el uso de bucles.

- *calculateErrorSplit*. Esta función será análoga a la desarrollada en el ejercicio anterior, cuyo objetivo es analizar el error para una posible partición, con la diferencia de que en el ejercicio anterior se evaluaba suponiendo que los modelos eran constantes. En este caso, se usan las salidas de un modelo de regresión para evaluar una posible partición. Por ese motivo, además de los argumentos que recibía la función en el ejercicio anterior, en esta recibe un cuarto argumento:
    - *modelOutputs*, de tipo *AbstractVector{<:Real}*, con las salidas del modelo de regresión para cada instancia del *dataset* (primer argumento)

  Esta función debe comenzar examinando si los valores de *modelOutputs* son todos iguales (función *unique*). En este caso, debe devolver una llamada a la función desarrollada en el ejercicio anterior (es decir, sin *modelOutputs*).

  En caso contrario, el desarrollo es similar a la del ejercicio anterior, con la diferencia de que, al llamar a la función *mse* para los dos subconjuntos ($x_j \leq t$ y $x_j > t$) las salidas que se toman no son los valores promedios de las salidas deseadas sino los valores de *modelOutputs* para ambos subconjuntos. Esta función debería devolver un valor de forma similar a la del ejercicio anterior, del mismo tipo que los datos del *dataset*. Para el desarrollo de esta función no se permite el uso de bucles.

- *bestSplit*. Al igual que la anterior, esta es similar a la del ejercicio anterior, pero en lugar de calcular las salidas como un promedio de las salidas deseadas, recibe un vector con estas salidas. Por lo tanto, además del *dataset* recibe un segundo argumento:
    - *modelOutputs*, de tipo *AbstractVector{<:Real}*, con las salidas del modelo de regresión para cada instancia del *dataset* (primer argumento)

  El desarrollo es similar a la del ejercicio anterior, con dos diferencias:
    - Cuando se calcula el MSE, no se hace con el promedio de las salidas deseadas, sino con el vector de salidas.
    - La llamada a *calculateErrorSplit* deberá incluir como último argumento adicional *modelOutputs*.

  Los valores a devolver deberán ser los mismos que los de la función del ejercicio anterior y, al igual que en esa función, para el desarrollo de esta función se permite hacer dos bucles,

para recorrer los atributos, y, interno a este, para recorrer los umbrales para cada atributo.

- *buildModelTree*. Nuevamente, esta es una función análoga a una del ejercicio anterior, llamada *buildRegressionTree*. En el ejercicio anterior se generaban modelos internos como constantes. En este caso, se generan modelos diversos. Por ese motivo, es necesario que esta función reciba, como segundo argumento, la configuración de los modelos internos. Por lo tanto, recibe dos argumentos:
    - *configuration*, de tipo *Dict*, con la configuración de los modelos internos a generar.
    - *dataset*, de tipo *Batch*, con el conjunto de entrenamiento.

  Además, al igual que en la función *buildRegressionTree*, esta función recibe como argumentos opcionales los hiperparámetros de este algoritmo contemplados en este ejercicio:
    - *max_depth*, de tipo *Int*, que contiene la profundidad máxima del árbol a generar. Su valor por defecto es -1, con lo que se indica que se generarán árboles de cualquier profundidad. Funciona como criterio para parar la recursividad.
    - *min_samples_leaf*, de tipo *Int*, que contiene el número mínimo de instancias del conjunto de entrenamiento que debe tener cada nodo. Al igual que el anterior, funciona como criterio parar parar el proceso recursivo. Su valor por defecto es 1.

  Esta función será casi igual a la función *buildModelTree*, con la única diferencia de que los modelos internos que se crean ya no son el resultado del promedio de las salidas deseadas, sino que se deberán crear mediante una llamada a la función *trainRegressionModel*. Por el resto, será igual a *buildModelTree*, incluyendo lo que debe devolver, y que no se pueden usar bucles.

  Es interesante tener en cuenta que, con estas funciones así definidas la construcción de un árbol de regresión en el ejercicio anterior constituye un caso particular de un árbol de modelos, y el mismo árbol de regresión se puede conseguir mediante una llamada a:

```
buildModelTree(dataset, Dict("modelType" => :constant))
```

- *trainRegressionModel*. Una nueva versión de esta función. Una vez se tiene la función para construir el árbol de modelos con la función *buildModelTree*, la función *trainRegressionModel* constituye un *wrapper* que permite usar los árboles de modelos mediante una interfaz común a todos los modelos de regresión utilizados. Al igual que en las anteriores, esta función recibe, como primer argumento, el tipo del modelo a construir (árbol de modelos), y los otros dos serán los mismos, es decir:

- El primer argumento será de tipo *Val{:ModelTree}*, que identifica que se va a entrenar un árbol de modelos.
- El segundo argumento será *configuration*, de tipo *Dict*, con la configuración a entrenar en las hojas y los nodos del árbol.
- El tercer argumento será *dataset*, de tipo *Batch*, con el conjunto de entrenamiento.

Esta función únicamente devolverá el resultado de una llamada a *buildModelTree*, pasándole como parámetros el *dataset* (segundo parámetro) y la configuración de los modelos internos del árbol (primer parámetro), no la configuración del árbol. Es importante tener clara esta distinción. El argumento *configuration* que recibe contiene la configuración del árbol de modelos. Los hiperparámetros que se van a utilizar por el árbol de modelos deberán estar dentro de esta variable *configuration*, y sus claves, que serán de tipo *String*, son las siguientes:

- "modelType", que identifica qué tipo de modelo se va a crear, y, para árboles de modelos, tendrá el valor *:ModelTree*
- "max_depth", con la profundidad máxima del árbol (por defecto, -1)
- "min_samples_leaf", con el número mínimo de instancias en cada hoja (por defecto, 1)
- "localModelConfiguration", que tendrá un nuevo *Dict* con la configuración de los modelos locales en nodos y hojas. Es este valor el que se deberá pasar como segundo argumento en la llamada a *buildModelTree*.

Además, en la llamada a *buildModelTree* se deberán especificar los dos argumentos opcionales, que se deberán leer del argumento *configuration* (claves "max_depth" y "min_samples_leaf"), y, si no están definidos en ese *Dict*, se tomarán los valores por defecto antes especificados.

Se puede ver así la flexibilidad para añadir un nuevo modelo de Regresión al conjunto de modelos ya disponibles, en este caso un Árbol de Modelos.

A continuación se muestra un ejemplo de cómo se podría entrenar un Árbol de Modelos en el que el modelo interno sea un SVR:

```
model = trainRegressionModel(
    Dict(
        "modelType"                => :ModelTree,
        "max_depth"                => 5,
        "min_samples_leaf"         => 10,
```

```
        "localModelConfiguration" => Dict(
            "modelType" => :SVR,
            "epsilon"   => 0.1,
            "C"         => 0.1,
            "kernel"    => "linear"
        )
    ), trainingDataset
)
```

Para el desarrollo de esta función no se permite el uso de bucles.

- *numInstances*. Esta es una función auxiliar que permite calcular cuántas instancias del conjunto de entrenamiento ha llegado a uno nodo u hoja concreto. Dado que se puede llamar sobre un árbol, es necesario sobrecargarla y definirla dos veces, según el único argumento, de nombre *tree*:
    - Si es de tipo *ModelTreeLeaf*, entonces se está llamando sobre una hoja. El número de instancias está almacenado en un campo de la propia hoja.
    - Si es de tipo *ModelTreeNode*, entonces se está llamando sobre un nodo. En esta implementación los nodos no almacenan el número de instancias, pero se puede calcular fácilmente haciendo la suma de los resultados de las llamadas recursivas de esta función para el hijo izquierdo y el derecho.

  En ambos casos, deberá devolver un valor entero positivo. Para el desarrollo de estas funciones sobrecargadas no se permite el uso de bucles.

- *predict*. Una vez construido el árbol de modelos, es necesario definir cómo utilizarlo para realizar predicciones sobre nuevos datos, para lo cual se dispone de la función *predict*, definida en el ejercicio anterior. En esa definición ya se tuvo cuidado en que la predicción en las hojas se realizase mediante una llamada a *predict* con el modelo almacenado (que, en ese caso, era una constante, pero podría ser cualquier otro tipo de modelo), por lo que es perfectamente válida aquí. Sin embargo, en este ejercicio se quiere ir un paso más allá, e implementar una función de predicción que incluya un suavizado en las predicciones, como se explica en clase de teoría, mediante un parámetro k. Al ser esta función aplicada sobre árboles, al igual que las anteriores, esta debe de estar sobrecargada y definida de forma separada para hojas y nodos. En ambos casos, la función recibe tres argumentos:
    - *tree*, de tipo *ModelTreeLeaf* o *ModelTreeNode*, con el árbol de modelos, que

puede ser una hoja o un nodo.

- *inputs*, de tipo *AbstractMatrix{<:Real}*, con la matriz de entradas.
- *k*, de tipo *Int*, con el valor del parámetro de suavizado en la predicción.

Para la función cuyo primer argumento es de tipo *ModelTreeLeaf*, la implementación es igual a la del ejercicio anterior, puesto que en las hojas no se realiza suavizado. De hecho, se puede implementar sencillamente llamando a *predict* con los dos primeros argumentos pasados (sin el tercero) para llamar a la función del ejercicio anterior.

En cambio, para la función cuyo primer argumento es de tipo *ModelTreeNode*, con un nodo interno del árbol, es necesario realizar el suavizado de la predicción obtenida de sus hijos. Para ello, se tiene el hiperparámetro *k*, que cuantifica la influencia del modelo en un nodo con respecto a la predicción obtenida de los hijos. Si *k* toma un valor de 0, quiere decir que el modelo en el nodo no tendrá ninguna influencia sobre las predicciones de los hijos. Por tanto, para ahorrar tiempo de cómputo, se debe comprobar primero si el valor de *k* es menor o igual a 0, y en este caso devolver el resultado de una llamada a *predict*, sin el valor de *k*, definida en el ejercicio anterior.

En caso contrario, el funcionamiento de esta función es parecido a la función del mismo nombre del ejercicio anterior: se parte la matriz de entradas *inputs* (función *splitDataset*) y cada parte se utiliza en una llamada a *predict* a cada hijo (con el valor de *k* como tercer argumento), pero, a la hora de asignar los resultados, será necesario combinarlas con las predicciones del modelo almacenado en este nodo, de la siguiente manera:

$$p' = \frac{n \cdot p + k \cdot q}{n + k}$$

donde p es la predicción que viene de un hijo, n es el número de instancias en ese hijo (se puede saber mediante la función *numInstances*), q es la predicción de este nodo, y k es la constante de suavizado.

Esta función deberá devolver, igual que el resto de *predict*, un vector cuyos elementos serán de tipo *Real*, sin ser necesariamente del mismo tipo que los elementos de *inputs*. Para hacer esta función no se permite el uso de bucles.

- *prune!*. Finalmente, la última función de este ejercicio se refiere a la poda de un árbol de modelos, que se realizará mediante un conjunto de validación. Dado que se realiza sobre un árbol, será una función sobrecargada, definiéndose de forma separada para nodos y para hojas. Esta función recibe dos argumentos:
  - *tree*, de tipo *ModelTreeLeaf* o *ModelTreeNode*, con el árbol de modelos, que

puede ser una hoja o un nodo.

- *validationDataset*, de tipo *Batch*, con el conjunto de validación.

Como siempre, la implementación más sencilla es para las hojas (tipo *ModelTreeLeaf*). En este caso, el resultado de la poda es la propia hoja; es decir, no se puede podar una hoja.

En cambio, para un nodo la poda se realizará siguiendo este orden:

1. En primer lugar, si el conjunto de validación está vacío, se devolverá el nodo actual. En caso contrario, será necesario partir el conjunto de validación de la manera habitual en un nodo ($x_j \leq t$ para el hijo izquierdo y $x_j > t$ para el hijo derecho), con la función *splitDataset*.
2. Se realizan dos llamadas recursivas a *prune!* para ambos hijos, con los subconjuntos de validación correspondientes, y se asignan los resultados a los hijos.
3. Se calcularán estos dos valores de error por medio de la función *mse*:
    - Error entre las predicciones de este nodo (función *predict* aplicada al nodo, que llama a los hijos que ya han sido podados, sin suavizado) en el conjunto de validación y las salidas deseadas del conjunto de validación.
    - Error entre las predicciones del modelo de este nodo (función *predict* aplicada al campo *model*) en el conjunto de validación y las salidas deseadas del conjunto de validación.
4. Finalmente, si el error de las predicciones del modelo de este nodo es inferior o igual al error del nodo, se devolverá una hoja, por medio de función *ModelTreeLeaf*, indicando como modelo de la hoja el modelo interno del nodo, y el número de instancias se puede calcular mediante la función numInstances.

Es importante tener en cuenta que el nombre de esta función termina por "!" (*bang*). Esto quiere decir que debe modificar el objeto pasado como argumento, es decir, el propio árbol. Adicionalmente, como resultado, esta función deberá devolver un objeto de tipo *ModelTree* con el árbol modificado, o sustituido, puesto que en alguna ocasión este árbol deberá ser sustituido por una hoja. Para el desarrollo de esta función no se permite el uso de bucles.

Firmas de las funciones:

A continuación se muestran las firmas de las funciones a realizar en los ejercicios propuestos. Tened en cuenta que, dependiendo de cómo se defina la función, esta puede contener o no la palabra reservada *function* al principio:

```
function predict(model::SymDoME.Tree, inputs::AbstractMatrix{<:Real})
function predict(model::LinearModel, inputs::AbstractMatrix{<:Real})

function trainRegressionModel(configuration::Dict, dataset::Batch)
function trainRegressionModel(::Val{:constant}, configuration::Dict,
                              dataset::Batch)
function trainRegressionModel(::Val{:linear}, configuration::Dict,
                              dataset::Batch)
function trainRegressionModel(::Val{:DoME}, configuration::Dict, dataset::Batch)
function trainRegressionModel(::Val{:SVR}, configuration::Dict, dataset::Batch)


function calculateErrorSplit(dataset::Batch, numFeature::Int, threshold::Real,
                             modelOutputs::AbstractVector{<:Real})



function bestSplit(dataset::Batch, modelOutputs::AbstractVector{<:Real})
function buildModelTree(configuration::Dict, dataset::Batch;
    max_depth        :: Int = -1,
    min_samples_leaf :: Int =  1
)

function trainRegressionModel(::Val{:ModelTree}, configuration::Dict,
                              dataset::Batch)


function numInstances(tree::ModelTreeLeaf)
function numInstances(tree::ModelTreeNode)

function predict(tree::ModelTreeLeaf, inputs::AbstractMatrix{<:Real}, k::Int)
function predict(tree::ModelTreeNode, inputs::AbstractMatrix{<:Real}, k::Int)

function prune!(tree::ModelTreeLeaf, validationDataset::Batch)
function prune!(tree::ModelTreeNode, validationDataset::Batch)
```